

Guía Definitiva de Testing en React

Arquitectura Integrada: Principios de Diseño, Estrategia de Consultas, Errores Comunes y Vitest

Propósito de este documento: Proporcionar una guía exhaustiva de aprendizaje para realizar pruebas en React combinando 5 fuentes clave de Testing Library y Vitest. Esta arquitectura abarca desde la filosofía rectora orientada al usuario hasta el manejo preciso de consultas (queries), las mejores prácticas de código y la infraestructura de ejecución con Vitest.

1. Nivel Filosófico: Principios Rectores de Testing Library

El principio fundamental expresado por Kent C. Dodds es: *"Cuanto más se parezcan tus pruebas a la forma en que se usa tu software, más confianza te darán"*. Las pruebas deben validar los resultados funcionales percibidos por el usuario final en lugar de inspeccionar la implementación técnica interna.

- Interacción con nodos del DOM:** Las pruebas deben trabajar directamente con los elementos del DOM que el usuario ve, en lugar de acceder a instancias de componentes o estados internos.
- Simulación de uso real:** Evaluar la aplicación a través de las interacciones que realizaría una persona (teclado, ratón) o una tecnología de asistencia (lectores de pantalla).
- APIs livianas y flexibles:** Mantener una suite de utilidades simple y comprensible sin sobrecargar el código de prueba.

2. Estrategia y Tipos de Consultas (About Queries)

Las *queries* son los métodos que ofrece Testing Library para localizar elementos en la página. Se dividen en tres familias según su comportamiento ante la presencia o ausencia de elementos:

Tipo de Query	0 Coincidencias	1 Coincidencia	>1 Coincidencias	Reintento Asíncrono
getBy...	Lanza Error	Retorna Elemento	Lanza Error	No
queryBy...	Retorna null	Retorna Elemento	Lanza Error	No
findBy...	Lanza Error	Retorna Elemento	Lanza Error	Sí (Promise / timeout)
getAllBy...	Lanza Error	Retorna Array	Retorna Array	No
queryAllBy...	Retorna []	Retorna Array	Retorna Array	No
findAllBy...	Lanza Error	Retorna Array	Retorna Array	Sí (Promise / timeout)

Orden de Prioridad Recomendado para Selección de Elementos

Para garantizar la accesibilidad y alineación con la experiencia del usuario, se debe seguir estrictamente el orden de prioridad definido en la documentación:

- 1. Consultas Accesibles para Todos:**
 - getByRole:** La máxima preferencia. Utilizar con la opción name (accesible name), e.g. `screen.getByRole('button', { name: /enviar/i })`.

- `getByLabelText`: Ideal para campos de formulario etiquetados.
 - `getByPlaceholderText`: Si no existe etiqueta formal.
 - `getByText`: Para contenido de texto interactivo o informativo fuera de formularios.
 - `getByDisplayValue`: Para elementos de formulario con valores cargados.
- **2. Consultas Semánticas de HTML5 / ARIA:**
 - `getByAltText`: Para elementos que soportan texto alternativo (imágenes, inputs de tipo imagen).
 - `getByTitle`: Atributo title (usar con precaución ya que no es leído consistentemente por lectores de pantalla).
 - **3. Test IDs (Último Recurso):**
 - `getByTestId`: Solo cuando el contenido es altamente dinámico o no existe una alternativa semántica explícita.

Uso de screen y Coincidencia de Texto (TextMatch)

Se recomienda usar la exportación **screen** para todas las consultas en lugar de desestructurar el retorno de `render`. `screen` está pre-vinculado a `document.body`. Además, la coincidencia de texto (`TextMatch`) acepta cadenas exactas, expresiones regulares (e.g. `/submit/i`) o funciones personalizadas. Por defecto, Testing Library **normaliza el texto** eliminando espacios al inicio/final y colapsando múltiples espacios intermedios en uno solo.

3. Buenas Prácticas y Errores Comunes (React Testing Library)

Basado en los patrones identificados por Kent C. Dodds, esta sección detalla las correcciones prácticas para evitar fallos frágiles o falsos positivos:

Práctica Incorrecta / Obsoleta (❌)	Práctica Recomendada (✅)	Razón del Cambio
Llamar manualmente a <code>cleanup()</code> en <code>afterEach</code>	No llamar a <code>cleanup</code>	<code>cleanup</code> ocurre de forma automática en los frameworks de prueba modernos.
Desestructurar <code>render</code> (e.g. <code>const { getByRole } = render(...)</code>)	Usar <code>screen.getByRole(...)</code>	Simplifica el código y no requiere actualizar imports al añadir o quitar queries.
Disparar eventos con <code>fireEvent.change</code>	Usar <code>userEvent.type(input, 'texto')</code>	<code>userEvent</code> dispara la secuencia real completa de eventos (<code>keyDown</code> , <code>keyPress</code> , <code>keyUp</code> , <code>change</code>).
Usar <code>queryBy*</code> para verificar existencia de elementos	Usar <code>getBy*</code> o <code>findBy*</code>	<code>queryBy*</code> sólo debe usarse para verificar no existencia (<code>not.toBeInTheDocument()</code>).
Envolver <code>render</code> o <code>fireEvent</code> en <code>act()</code> manualmente	Llamar directamente a <code>render</code> o <code>fireEvent</code>	Las utilidades de Testing Library ya están envueltas internamente en <code>act()</code> .
Envolver <code>getBy*</code> dentro de <code>waitFor(...)</code>	Usar la variante asíncrona <code>await screen.findBy*</code>	<code>findBy*</code> integra <code>waitFor</code> nativamente con mejores mensajes de error.
Colocar múltiples aserciones o efectos secundarios dentro de <code>waitFor</code>	Una sola aserción por callback de <code>waitFor</code>	Evita que efectos secundarios se ejecuten múltiples veces si el callback reintenta.
Aserciones genéricas como <code>expect(button.disabled).toBe(true)</code>	Usar <code>@testing-library/jest-dom</code> (<code>expect(button).toBeDisabled()</code>)	Produce mensajes de error mucho más claros y descriptivos en caso de fallo.

4. Infraestructura de Ejecución: Vitest Getting Started

Vitest es un marco de pruebas de última generación impulsado por el ecosistema Vite. Proporciona una alternativa de ultra alto rendimiento que reutiliza la configuración existente del proyecto.

- **Instalación y Scripts:** Se instala como dependencia de desarrollo mediante `npm install -D vitest` y se configura en `package.json` con `"scripts": { "test": "vitest" }`.
- **Sintaxis nativa:** Importa utilidades globales como `test` y `expect` desde `'vitest'`.
- **Integración con Vite:** Lee automáticamente `vite.config.*` o un archivo dedicado `vitest.config.*` para la simulación del DOM (JSDOM o Happy-DOM).
- **Modos de ejecución:** Ejecuta en modo de observación continua (**watch mode**) por defecto. Para ejecuciones únicas en integración continua (CI), se utiliza `vitest run`.

Ejemplo de Código Integrado (React + Vitest + Testing Library):

```
import { render, screen } from '@testing-library/react' import userEvent from '@testing-library/user-event'
import { test, expect } from 'vitest' import { LoginForm } from './LoginForm' test('permite al usuario
iniciar sesión correctamente', async () => { render(<LoginForm />) // 1. Selección por Rol Accesible const
userInput = screen.getByRole('textbox', { name: /usuario/i }) const submitBtn = screen.getByRole('button',
{ name: /ingresar/i }) // 2. Simulación de Interacción Real await userEvent.type(userInput, 'maria_dev')
await userEvent.click(submitBtn) // 3. Manejo de Asincronía y Aserción Accesible const welcomeMsg = await
screen.findByRole('heading', { name: /bienvenida maria_dev/i }) expect(welcomeMsg).toBeInTheDocument() })
```

5. Capa de Extensibilidad Avanzada: Programación con Vitest Node API

Para creadores de librerías, entornos de CI personalizados o herramientas de desarrollo, Vitest ofrece APIs programáticas avanzadas en Node.js que permiten controlar el ciclo de vida del runner:

Diferencias Principales entre startVitest y createVitest

Método API	Validación de Paquetes	Ejecución Inmediata	Casos de Uso Principales
startVitest	Sí (Valida instalaciones necesarias)	Sí (Ejecuta pruebas de inmediato)	Scripts de automatización directa, herramientas rápidas CLI.
createVitest	No (Requiere inicialización manual)	No (Crea la instancia en reposo)	Integraciones complejas, herramientas con ciclo de vida extendido, watchers personalizados.

Al utilizar `createVitest`, la instancia no se cierra automáticamente. Es necesario invocar `await vitest.init()` para inicializar los **reporters** y proveedores de cobertura, y controlar eventos como `vitest.onCancel()`, `vitest.onClose()` y re-ejecuciones manuales mediante `vitest.rerunTestSpecifications(specifications)`.

6. Matriz de Conexión entre las 5 Fuentes de Aprendizaje

Fuente	Nivel de la Arquitectura	Aporte Clave al Testing en React
Guiding Principles Testing Library	Filosofía y Mente	Establece que las pruebas deben imitar el uso real e interactuar con el DOM, no con componentes internos.
About Queries Testing Library	Selección de Elementos	Define la semántica y jerarquía de búsqueda accesible (getByRole, findBy*, queryBy*, screen).
Common Mistakes RTL	Práctica y Refactorización	Elimina vicios como cleanup manual, uso de fireEvent, abuso de act o aserciones incorrectas.
Getting Started Vitest	Infraestructura Runner	Proporciona el entorno de ejecución ultrarrápido integrado con Vite, test y expect.
Running Tests Advanced Vitest	Automatización Extensible	Permite controlar programáticamente el motor de pruebas desde scripts Node.js mediante startVitest y createVitest.

Conclusión del Aprendizaje: Dominar el testing en React requiere integrar estas 5 capas: pensar en el usuario (Principios), consultar el DOM de forma accesible (About Queries), escribir código limpio y sin errores (Common Mistakes), ejecutar velozmente en el ecosistema Vite (Getting Started Vitest) y extender el motor cuando sea necesario (Running Tests Advanced).